Pragmatic DDD Guide

Oct 28, 2026 · @kodebot

This guide explains how we write code inside our Spring Boot microservices. We use a light form of Domain-Driven Design (DDD). The guide has two aims:

- **Every business rule is written in exactly one place.** No code can change data without going through that rule.
- **A design mistake can be fixed inside one part of one microservice.** Other microservices do not need to change.

The reference code is the `manage-claim-service` repository. If this guide does not answer your question, copy what the reference code does.

Technology: Java 21, Spring Boot 4, Spring Data MongoDB and Amazon DocumentDB. DocumentDB is a database that accepts MongoDB commands, but it behaves differently from MongoDB (see section 6). We also use our shared `service-platform` library, and Amazon SNS and SQS to send and receive events.

Terms used throughout

Term	Meaning
Microservice	An application that is deployed on its own and has its own database. Example: `manage-claim-service`
Feature	One business area inside a microservice. Each feature has its own folder. Example: `claim/` or `note/`
Business action	One change that the user asks for with one button. Examples: "close claim", "change address". Each business action is one application service method
Platform library	Our shared `service-platform` library. Every microservice uses it. It provides `AggregateRoot`, `Guard`, `ResilientCalls`, the outbox and the inbox
Design group	The lead engineer, two or three senior engineers and the business experts. They approve the design decisions in section 4
Design note	A one-page description of a feature. The design group approves it before coding starts. The template is in section 4.4
Migration	A Java class that changes the database. For example, it creates an index, loads lookup data or fixes data. Each migration runs once, automatically, when the microservice starts

Other terms are explained where they first appear. They are also listed in the glossary (section 11).

1. The example used in this guide

A **caseworker** records a **claim** for a person. This person is the **claimant**. The caseworker adds **evidence** to the claim. Evidence is a document, such as a payslip. Later, the caseworker marks each evidence item as verified.

The caseworker also writes **notes** about the claim. A note can be marked "needs action", for example "call back on Monday". It stays like this until someone marks the action as done. At the end, the caseworker closes the claim.

Two microservices from other teams are involved:

- The **eligibility** microservice says whether a person may claim.
- The **payment** microservice pays claimants. It tells us when it has paid.

The business rules:

1. A claim can be created only if the eligibility microservice says the claimant is eligible. The claim type must also be active (Standard or Urgent).
2. A claim has at most 10 evidence items. Each item can be verified only once.
3. A claim can be closed only when all its evidence is verified and no note on it still needs action.
4. A closed claim cannot be changed.
5. Messages can arrive late or twice. So the claim's last payment is replaced only by a payment with a later payment date.
6. Entitlement is the amount owed for a period. Entitlement = weekly rate for the claim type × days the claim was open in the period ÷ 7. Count both the first day and the last day. Count days as dates in UK time. Round half-even to the penny. Example: the rate is £100 a week. The claim opened on 15 January. The period is 1–31 January. The claim was open for 17 days. So £100 × 17 ÷ 7 = £242.86.

How this is stored: each claim is one document in the `claims` collection. Its evidence items are stored inside that document. Each note is a separate document in the `notes` collection. A note holds the ID of its claim. Section 4.3 explains why evidence is inside the claim but notes are not.

2. The building blocks

DDD means two things. Write code in the business's own words. Put each business rule inside the object it is about. Compare two ways to close a claim:

```
// Without DDD: the rule sits in a service class, and any code can change the status
if (claim.getEvidence().stream().anyMatch(e -> !e.isVerified())) {
    throw new RuntimeException("Unverified evidence");
}
claim.setStatus("CLOSED");

// With DDD: there is one way to close a claim, and it checks every rule itself
claim.close(notesNeedingAction, clock);
```

In the first version, other code that closes a claim must remember to copy the check. A nightly job is one example. In the second version, other code cannot skip the check.

Use the business's words for classes, methods and exceptions. Use `close`, `verifyEvidence` and `ClaimHasOpenActionsException`. Do not use `updateStatus` or `ValidationException`.

These are the building blocks. Do not use any other building block without design group approval.

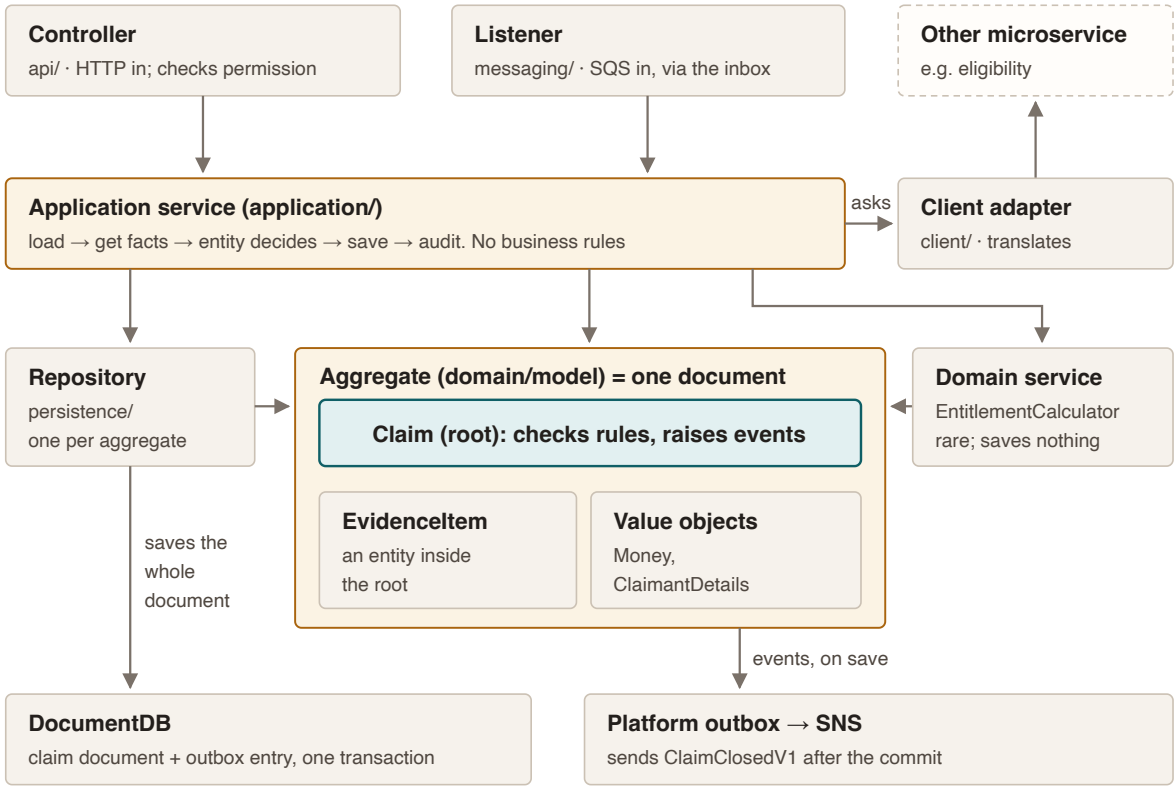
Building block	What it is	How we write it	Example
Entity	A business object with its own ID. It keeps the same identity when its details change	A class with a `String` UUID as its ID. It has private fields and no setters. Its fields change only inside its own business methods	`Claim`, `Note`
Value object	A value with no ID. Its contents define it. Two `Money` objects of £100.00 are equal. (`Money` always keeps two decimal places, so £100.0 is stored as £100.00)	A Java `record`. It cannot change after it is created. Its constructor checks its values. Use a value object only when a value has rules (money cannot be negative), or when several fields belong together (name and date of birth)	`Money`, `ClaimantDetails`
Invariant	A business rule that must be true at all times, not only when someone checks it	Write it in the design note with an example. Enforce it in an entity method	"A claim has at most 10 evidence items"
Aggregate and **aggregate root**	A group of objects that are always loaded and saved together, as one document. The root is the top object. All changes go through the root	The root extends `AggregateRoot` from the platform library and has `@Document`. Code outside the aggregate never changes the inner objects directly	`Claim` (root) with its `EvidenceItem`s
Factory method	The only way to create a new entity. It makes sure the entity is valid from the start	A `static create(...)` method on the entity. It checks the rules for creating the entity	`Claim.create(type, claimant, eligibility, clock)`
Repository	The interface that loads and saves whole aggregates	One Spring Data interface for each aggregate root. Only its own feature uses it	`ClaimRepository`
Domain event	A record that something happened. Its name is in the past tense	The root's method creates it with `raise(new ClaimClosed(...))`. When the root is saved, the feature's event mapper can turn it into an integration event for other microservices (section 7)	`ClaimClosed`
Domain exception	An error that names the business rule that was broken	It extends one of three platform classes. The class sets the HTTP status: `NotFoundException` (404), `BusinessRuleViolationException` (422) or `InvalidValueException` (400)	`ClaimClosedException`
Domain service	A business calculation that needs several objects and does not belong to any one of them. It is rare	A plain class that stores no data. It takes its inputs and returns a result. It does not load or save anything	`EntitlementCalculator`

Building block	What it is	How we write it	Example
Application service	The class that runs each use case, step by step	One for each feature. It loads data, calls the entity, saves, and writes the audit record. It contains no business rules and no permission checks	`ClaimService`
Client adapter	The only class that talks to another microservice or another system. It translates their data into our types, and their errors into our exceptions. (DDD calls this an anti-corruption layer)	It lives in `client/`. It returns our own types	`EligibilityClient`

Which rules are always true, and which are checked at a moment

- **Inside one aggregate, rules are always true.** The whole document is saved in one transaction. Each document has a version number. If someone else saved the document after you loaded it, your save is refused with `409 Conflict`.
- **Between two aggregates, a rule is true only at the moment it is checked.** For example, when a claim is closed, the application service counts the notes at that moment. It does not see a note that is added one second later. Other microservices learn about changes a little later, through events.

So, if a rule must be true at all times, everything the rule needs must be inside one aggregate. If it is enough to check the rule at one moment, separate aggregates are fine. The design note says which one applies.



How the building blocks handle one request: the application service runs the steps, and the aggregate decides. A controller or a listener calls the application service. It asks other microservices through a client adapter, loads and saves through the repository, and uses a domain service for a calculation. The Claim root checks the rules and raises events. The repository saves the whole document, with its outbox entry, to DocumentDB in one transaction, and the platform outbox sends the events to SNS after the commit.

How a change flows through the code:

1. A request arrives at a controller, or a message arrives at a listener.
2. The controller or listener calls the application service.
3. If the change needs an answer from another microservice, the application service asks for it through a client adapter. It does this first, before the transaction starts.

4. The application service loads the aggregate through its repository.
5. It calls a method on the aggregate root. The method checks the rules and makes the change.
6. The application service saves the aggregate. In the same transaction, the platform stores the integration events in the outbox. The outbox is a collection that holds events until they are sent (section 7.2).
7. After the transaction, the platform sends the events.

3. The ten golden rules

Rules 1, 3 and 4 apply to Rich features (section 4.2). All other rules apply to every feature. In the "Checked by" column, "Build" means an architecture test fails the build if the rule is broken. "Review" means the reviewer checks the rule.

#	Rule	Example	Why	Checked by
1	An entity's fields change only inside its own methods. Entities have no public setters	`claim.close(...)`, never `claim.setStatus(CLOSED)`	The method checks the rules and records the event. A setter skips both	Build
2	The aggregate root class is also the stored document. It has `@Document` and extends `AggregateRoot`. Do not write a second class for the database	`@Document("claims") class Claim extends AggregateRoot`	A second class and a mapper double the code. DocumentDB does not need them	Build
3	A rule that decides if a change is allowed goes in the entity method that makes the change. If the rule needs information that the entity does not hold, the application service gets the information and passes it in as a method parameter	The claim does not hold its notes. So the application service counts them: `claim.close(notesNeedingAction, clock)`	All the rules for one change are in one method. The parameter is required, so nobody can forget the check	Review
4	Application service methods contain only steps, never business decisions. Section 5.3 lists the steps	No `if (claim.status() == CLOSED)` in `ClaimService`	Rules in services get copied to several places. Then the copies become different	Review
5	A transaction saves one aggregate. To save more than one aggregate in one transaction, you need approval in the design note	Creating a claim with its first notes is approved. Nothing else is approved	Small transactions have fewer conflicts and stay within DocumentDB's limits	Review
6	An aggregate refers to another aggregate only by its ID, as a `String`	`Note` has `String claimId`, not a `Claim` field	If it holds the object, both are loaded and saved together. This silently makes one large aggregate	Review
7	A feature uses another feature only by calling its application service. It passes and receives simple values (IDs, numbers, small records), never entities	`ClaimService` calls `notes.countNeedingAction(claimId)`, never `NoteRepository`	Each feature can then change how it stores its data without breaking the others	Build
8	Feature code saves only with the repository's `save`, `saveAll` or `delete`. It never updates documents with `MongoTemplate`. Migrations are the only exception	`claims.save(claim)`	Saving through the repository sends the entity's events. A `MongoTemplate` update changes data and sends no events	Build
9	Check permissions on the controller. Every endpoint has `@ClaimsRead` or `@ClaimsWrite`. Application services do not check permissions	`@PostMapping("/{claimId}/close")` `@ClaimsWrite`	The controller knows the user, because the request arrives there. Message listeners also call application services, and there is no user there	Build, plus a test that expects `403`
10	Reach other microservices and systems only through a client adapter, or through versioned events. Wrap every call in `ResilientCalls` (section 7.4)	`EligibilityClient`, `CustomerRecordClient`	When another team changes its API, only the adapter changes	Build and review

4. Designing a feature

4.1 Who decides

Do not write code for a feature with business rules until its design note is approved.

Decision	Who decides	Where it is written
Which microservices exist, and what each one owns	The design group, with the business	An architecture decision record (ADR) in Confluence
What each aggregate contains, the business rules with examples, and any transaction that saves two aggregates	The design group	The feature's design note
Endpoints, classes and tests	The engineers who build the feature	The code

4.2 Simple or Rich

Each design note says which level the feature uses:

- **Simple** is for data with no business rules, such as lookup lists or user preferences. The entity has fields and getters. Checks can go in the application service. Copy the `reference/` feature. A Simple feature can also have no entity at all, when another system stores the data (the `customer` feature).
- **Rich** is for data with states and rules, such as claims, payments and awards. Everything in this guide applies. Copy the `claim/` feature.

Core business areas are Rich. If a Simple feature gets business rules, it becomes Rich.

4.3 Inside the aggregate, or a separate aggregate?

Some objects belong to another object. For example, evidence and notes both belong to a claim. For each one, answer four questions:

1. Does a rule on the parent need its full details, not only a count?
2. Is there a fixed maximum number of them, written in the design note? (A DocumentDB document cannot be bigger than 16 MB. So a list with no maximum will fail one day.)
3. Is it changed at the same time as the parent, by the same people?
4. Is it meaningless without the parent?

If all four answers are yes, put it **inside** the aggregate. If any answer is no, make it a **separate aggregate**. The separate aggregate stores the parent's ID.

Example: evidence is inside the claim. Closing a claim needs all the evidence, and a claim has at most 10 items. Notes are a separate aggregate. There is no limit on notes. Closing needs only a count. Caseworkers add notes all the time. If notes were inside the claim, two caseworkers could add notes at the same time. Then one of them would get a `409` error.

Do not design aggregates by copying mainframe tables. The tables were built for a different system.

4.4 Design note template

```
# Feature: <name>            Level: Rich | Simple
Owner: <name>      Approved by: <design group member>      Date: <date>

## Purpose
One or two sentences in the business's words.

## Aggregates
| Aggregate | What is inside it | Maximum size | Collection |
Separate aggregates and the ID they hold: ...

## States
Example: OPEN -> CLOSED (close). CLOSED is final.

## Rules
| Rule | Where it is enforced | Example with real values |

## Events
| Event | Sent to other microservices as | Who uses it |

## Talks to
| Feature or microservice | Why | How |

## Approved exceptions to this guide
## Open questions for the business
```

5. Building a feature

Start by copying `claim/` (Rich) or `reference/` (Simple) from the reference code. Rename it. Delete what you do not need.

5.1 Folders

Folder	What goes in it
`domain/model`	Entities (the classes with `@Document`), the objects stored inside them, and value objects
`domain/service`	Domain services, if there are any. Domain services have no Spring annotations. So register each one as a bean in the feature's `...Config` class
`domain/event`	Domain events
`domain/exception`	Domain exceptions
`application`	The application service
`persistence`	The Spring Data repository
`api`	The controller, the request and response classes (DTOs), and the code that turns an entity into a response
`client`	Client adapters for other microservices, and their request and response classes
`messaging`	The event mapper, the published event classes, and the listeners for other microservices' events. The event mapper decides which events other microservices receive

Use only the folders that the feature needs. For example, the `customer` feature has no `persistence` folder, because the central customer service stores its data.

5.2 Where does this code go?

Answer these questions in order. Stop at the first "yes".

1. **Does the code decide if the entity is allowed to change, based on its current state?** Put it in that entity's method. (Format checks are question 4. Codes from another feature are question 5.) If the code needs information that the entity does not hold, add a method parameter. The application service then passes the information in.
2. **Is it a business calculation that combines several things, belongs to no single entity, and changes nothing?** Put it in a domain service, such as `EntitlementCalculator`. Do this only if you can write a table of worked examples with real numbers. If the logic is one or two `if` statements, put it in the entity (question 1).
3. **Does it load, save, call another microservice, start a transaction or write an audit record?** Put it in the application service.

4. **Does it check the format of a single value?** For example: not blank, a valid date, not negative. Put it in the value object's constructor. Or use `Guard` in the entity's `create` method. `Guard` is the platform's one-line check helper.
5. **Does it check that a code exists in another feature, such as a valid claim type?** Call that feature's application service: `referenceData.requireActive(...)`.

5.3 The six steps of an application service method

Every application service method that changes data follows these steps, in this order:

```
@Transactional
public void close(String claimId, Long expectedVersion) {
    Claim claim = claims.get(claimId);                          // 1. Load the aggregate (404 if it doesn't exist)
    claim.requireVersion(expectedVersion);                      // 2. Check the user saw the latest version (409 if not)
    long notesNeedingAction = notes.countNeedingAction(claimId); // 3. Fetch any information the rule needs from
elsewhere
    claim.close(notesNeedingAction, clock);                     // 4. Call the entity method; it checks the rules and
changes
    claims.save(claim);                                         // 5. Save; this also sends the entity's events
    audit.record(AuditActions.CLAIM_CLOSED, AuditActions.CLAIM, claimId); // 6. Write the audit record
}
```

- If the method needs an answer from another microservice, get it **before** the transaction starts. A slow call must never keep a database transaction open. `ClaimService.create` shows how: it calls the eligibility microservice first, and then saves inside a `TransactionTemplate`.
- Get the current time from the injected `Clock`. Never use `Instant.now()`. Then tests can set the time.
- Some business actions only send a change to another system, such as "change address". They have no load, no save and no transaction. They make one call, and then write the audit record (section 7.4).

5.4 Tests

What you wrote	Test it with	What the test proves
Entity, value object, domain service	Plain JUnit, with no Spring and no mocks	Each business rule works, using the examples from the design note
Application service	JUnit, with Mockito mocks for the repository and other features	The steps happen in order. Nothing is saved when a rule fails. The saved entity has the right events (read them with `pullEvents()`)
Controller	`@PlatformWebMvcTest`	Status codes, validation errors, and `403` when the user has no permission
Client adapter	JUnit with a local fake HTTP server	Their answers become our types and our exceptions. For a read, a `503` is retried and a `4xx` is not. For a change, nothing is retried, and a timeout gives `504`
Repository, and the whole microservice	Tests named `*IT`, with a real database in Docker	Saving and loading keep every field. Saving with an old version throws `OptimisticLockingFailureException`. Queries work. The expected events and audit records are stored

5.5 Worked example: "a closed claim can be reopened within 28 days"

1. **Design note:** add the state change `CLOSED -> OPEN (reopen)`. Add the rules with examples. For example, a claim closed on 1 March can be reopened on 29 March, but not on 30 March. Add the event `ClaimReopened`, which is sent to other microservices as `ClaimReopenedV1`. Change business rule 4 to "A closed claim cannot be changed, except by reopening it within 28 days". Count days as dates in UK time. Get the design note approved.
2. **Domain:** create the exceptions `ClaimNotClosedException` and `ReopenWindowPassedException`. Both extend `BusinessRuleViolationException`. Create the event record `ClaimReopened`. Write the entity method:

```
private static final ZoneId UK = ZoneId.of("Europe/London");

public void reopen(String reason, Clock clock) {
    Guard.notBlank(reason, "reason");                                  // 400 if blank
    if (status != ClaimStatus.CLOSED) throw new ClaimNotClosedException(id());
    Instant now = clock.instant();
    LocalDate closedOn = LocalDate.ofInstant(closedAt, UK);            // count days as UK dates
    LocalDate today = LocalDate.ofInstant(now, UK);
    if (today.isAfter(closedOn.plusDays(28))) throw new ReopenWindowPassedException(id());
    status = ClaimStatus.OPEN;
    closedAt = null;
    raise(new ClaimReopened(id(), reason, now));
}
```

3. **Domain tests:** write one test for each example in the design note. Make them pass before you do anything else.
4. **Application service:** add `reopen(claimId, reason, expectedVersion)`. It loads the claim and checks the version with `claim.requireVersion(expectedVersion)`. Then it calls `claim.reopen(reason, clock)`, saves the claim, and records the audit action `CLAIM_REOPENED`. Add an application service test.
5. **API:** add `POST /claims/{claimId}/reopen` with `@ClaimsWrite`. Its request class is `ReopenClaimRequest(@NotNull Long version, @NotBlank String reason)`. Test `204`, `403`, `409`, `422`, `400` and `404`. Update `contracts/openapi.yaml`.
6. **Messaging:** add the `ClaimReopenedV1` record. Add a line in the event mapper that turns `ClaimReopened` into `ClaimReopenedV1`. Add its JSON schema and a mapper test. Do not write any code to send the event. Saving the claim sends it.
7. **Database:** there is no new query. So you do not need a new index or migration.
8. **Pull request:** the reviewer checks the change against the golden rules (section 3) and the DocumentDB rules (section 6).

6. DocumentDB rules

Amazon DocumentDB accepts MongoDB commands, but it has its own limits. These rules come from the AWS documentation: transactions, retryable writes, differences from MongoDB.

Rule	Why
A transaction must finish well within one minute. Never call another microservice inside a transaction. Never loop over many records inside a transaction	DocumentDB cancels any transaction that runs for more than one minute. You cannot change this limit
A transaction saves one aggregate, or the few aggregates that the design note approves. A job that updates many records runs one transaction for each record	One transaction cannot change more than 32 MB of data
A migration creates every collection before any code uses it	DocumentDB cannot create a collection inside a transaction
Inside a transaction, keep reads small. Load documents by ID, or run a count. Never read a long list	DocumentDB does not support cursors inside a transaction. A cursor is how a large, paged result is read
Start transactions only with `@Transactional` or `TransactionTemplate`	The platform library sets up transactions correctly for DocumentDB
Do not write your own retry loops around saves	DocumentDB versions before 8.0.2 do not support retryable writes, so the platform sets `retryWrites=false`. The database never retries a failed write. The caller retries instead: the user tries again, or SQS delivers the message again
Never remove the version check, and never work around it. A save with an old version is refused with `409`	Without the check, a second user's save silently overwrites the first user's change
Every change must be safe to repeat. A repeated request or message must never add a list item, a document or an amount a second time (for example, `count = count + 1`)	After a timeout or a database restart, a saved change can be reported as failed. Then it is sent again. Three things stop the repeat from having an extra effect: the version check, the `Idempotency-Key` header on create requests (the platform returns the first result for a repeated key), and the inbox for messages (section 7.3)
Always read from the primary database instance (`readPreference=primary` in the connection string)	Copies on other instances can be slightly behind. You might not see a change you just made
Every query that can return more than one document must sort explicitly. Every query must use an index that a migration created, or the built-in `_id` index	Without a sort, DocumentDB returns results in no fixed order. DocumentDB does not create indexes automatically

Rule	Why
Do not use `$regex` (pattern matching), `$lookup` (joining collections) or unusual aggregation operators without design group approval	DocumentDB supports them only partly. They often cannot use indexes
Never edit a migration that has already run in any environment. To change something, add a new migration. Write every migration so that it is safe to run twice	If you do not, environments end up with different data
Before you add an index to a large collection, test how long it takes on production-sized data. Agree the release time with the platform team	DocumentDB builds only one index per collection at a time, and building takes time
Store money with the `Money` value object, which is stored as an exact decimal. Store times as `Instant`, in UTC. When a rule counts days, convert the times to UK dates (`Europe/London`) first. Give every list inside a document a maximum size	This avoids rounding errors and daylight-saving mistakes. It also keeps documents under the 16 MB limit

Open question for the platform team: AWS says that DocumentDB does not support "causally consistent" sessions. The pilot must confirm that Spring transactions work with our DocumentDB version.

7. Events and other microservices

7.1 Two kinds of event

- A **domain event** (such as `ClaimClosed`) is used only inside our microservice. We can change it at any time.
- An **integration event** (such as `ClaimClosedV1`) is the message that other microservices receive. It has a JSON schema in `contracts/events/`. Other teams depend on it, so we can only add fields to it. To remove or rename a field, create a new version (`ClaimClosedV2`). Send the new version together with V1 until every receiving team has moved to V2.

Each feature has an **event mapper**. It decides which domain events become integration events. Most domain events are never sent outside the microservice.

7.2 How we send events (the outbox)

1. The entity method records a domain event with `raise(...)`.
2. The application service saves the entity with the repository.
3. The platform library asks the feature's event mapper which integration event to send for each domain event. Most domain events have none. The platform stores each integration event in the `event_publication` collection, **in the same transaction** as the entity. This collection is called the **outbox**.
4. After the transaction commits, the platform sends the event to SNS. If SNS is not available, the platform tries again later.

So a saved change always sends its integration events. A change that was not saved never sends them. Never send to SNS in any other way.

7.3 How we receive events (the inbox)

- Each listener extends the platform's `InboxMessageHandler`. The **inbox** is a list of event IDs that were already processed. Before the listener handles a message, it checks the inbox and skips duplicates. It uses the `eventId` field inside the event, not the SQS message ID. This is because a resent event arrives as a new SQS message. The listener records the event ID in the same transaction as the change.
- A listener only translates the message into our types and calls an application service. It contains no business rules.
- If handling fails, SQS delivers the message again. After 5 failed attempts, the message moves to a **dead-letter queue**. Someone must investigate it there.
- Messages can arrive late or in the wrong order. So entity methods must handle this. For example, `Claim.recordPayment` ignores a payment that is older than the one it already has.
- Our copies of other teams' events ignore fields that we do not know. So when other teams add fields, our code does not break.

7.4 Calling another microservice or system

- Call another microservice only when you need its answer before you can continue. For example, you need eligibility before you create a claim. In other cases, use events.

- Make every call through a client adapter. Create its HTTP client with the platform's `HttpClientFactory`, which adds time limits. Wrap each call in `ResilientCalls`:
    - **Reads and checks** use `callWithRetry`. It retries timeouts and server errors. If they keep failing, our endpoint returns `503`.
    - **Changes** use `callOnce`. It never retries. If we could not reach the other system, it returns `503`, and nothing changed. If the call timed out after it was sent, it returns `504`. Then we do not know if the change was made, so the user must check before trying again.
    - Both use a **circuit breaker**. After repeated failures, a circuit breaker stops calling for a short time. So we do not keep waiting for a failing microservice.
- Keep the total time for one request, with all retries, under 4 seconds. The front end waits at most 5 seconds, so our calls must give up first.
- Make the call before the database transaction starts.
- A business action can send its change to another system. For example, "change address" sends the new address to the central customer service. Make exactly one call for the business action. Do not save anything else in the same request. If we keep a copy of the data, update the copy only after the other system confirms the change.
- Every adapter has a client test. It also has a WireMock stub in the `local-environment` repository, for local development. A WireMock stub is a fake version of the other microservice.

7.5 Changes that span several microservices

A database transaction cannot cover two microservices. Some business processes must change data in several microservices. Design such a process as a series of steps. Each step changes one microservice. Then it sends an event that starts the next step. If a step fails, a step that was agreed in advance deals with it. For example, the action can be "create a task for a caseworker". Every process like this needs design group approval and an ADR.

8. Fixing mistakes

Mistakes will happen. You can fix them safely, for two reasons. First, each feature's data is reached only through its application service (golden rule 7). Second, other microservices see only our versioned events. So you can correct a wrong design inside one feature. You change that feature's code and add a migration.

For every mistake:

1. Do the step in the "Do now" column. It is small and safe to release quickly.
2. Tell the design group on the same day.
3. Create a ticket for the proper fix, with a date. Note the mistake in the feature's design note.
4. Do the step in the "Fix properly" column by that date.
5. Add an architecture test or a unit test. Then the same mistake fails the build next time.

Design mistakes

Mistake	How you notice	Do now	Fix properly	Never
Two separate aggregates share a rule that must always be true	Data breaks the rule. Or code keeps saving both aggregates together	Save both in one transaction, in one application service method. Add this to the design note as an exception, for the design group to approve	Merge them into one aggregate. A migration moves the child documents into the parent document	Try to keep the rule true by sending events between them. Events arrive later, so the rule is broken in between

Mistake	How you notice	Do now	Fix properly	Never
An aggregate is too big	Users often get `409 Conflict`. Documents keep growing	Keep the version check. Make sure the screen tells the user to reload	Move the part that changes often, or that has no maximum, into its own aggregate. The new aggregate holds the parent's ID. If a parent rule needs information about it, the application service passes the information in as a parameter	Turn off the version check. Or update single fields with `MongoTemplate` to avoid the conflict
A list inside a document has no maximum	Some documents are much larger than others	Add a maximum in the entity method that adds items. Refuse more items with a `BusinessRuleViolationException` (`422`). Add an alert on document size	Move the list into its own aggregate	Wait until a document reaches DocumentDB's 16 MB limit and writes start to fail
A business rule is in an application service, a controller or a listener	There is an `if` about status, amounts, dates or counts outside `domain/`	Create a ticket for this sprint	Move the rule into the entity method. Add a parameter if the rule needs outside information. Add a unit test	Copy the same `if` to another place
The same rule is written in several places	Two places check the same thing in slightly different ways	Agree with the business which version is correct	Keep one copy, in the entity method. Delete the others	Keep both copies "to be safe"
A class exists only to hold one or two `if` statements (such as `ClaimClosurePolicy`)	A tiny "policy", "validator" or "helper" class	Nothing urgent	Move the check into the entity method that it protects. Delete the class	Add more classes like it

Mistake	How you notice	Do now	Fix properly	Never
An entity is changed without its methods. For example, someone added a setter, or a script writes fields directly	The architecture test fails. Or data appears that the rules do not allow. Or other microservices miss changes	Remove the setter or stop the script. Find the affected documents	Make scripts load each entity and call its method. Repair wrong data in the same way, so that rules, events and audit all apply	Repair data by writing directly to the database, except in a reviewed migration
A feature uses another feature's repository or entity	The architecture test `featuresUseEachOtherOnlyThroughApplication` fails	If the release cannot wait, record today's violations with ArchUnit's `FreezingArchRule`. Then only new violations fail the build. Create a ticket	Add a method to the other feature's application service that returns a simple value. Call that method	Turn the architecture test off
Two features call each other, so Spring needs `@Lazy` to start	`@Lazy` appears in more than one place	One `@Lazy` is acceptable if the design group approves it	Make the calls go in one direction only. Use events for the other direction. Or merge the two features	Add `@Lazy` without asking

Data and DocumentDB mistakes

Mistake	How you notice	Do now	Fix properly	Never
A query has no index	The endpoint gets slower as data grows. `explain()` shows that the whole collection is read	Add the index in a new migration. If the collection is large, agree the release time	During review, check that every new query has an index	Edit an old migration. Or create an index by hand in production
A migration was wrong, or someone edited it after it ran	Environments have different data or indexes. Or startup fails	Stop deploying. Put any edited migration back exactly as it was	Write a new migration that corrects the data. Make it safe to run twice	Edit or delete rows in the `schema_migrations` collection by hand
A transaction times out, or it contains a call to another microservice	Errors say that the transaction was aborted, mostly under load	Move the call to before the transaction	Make each transaction save one aggregate, with no loops	Try to raise DocumentDB's one-minute limit. It cannot be changed
A job updates many records in one transaction	The job fails part of the way through, and nothing is saved	Stop the job	Process one record per transaction, by calling the application service for each record. Make the job skip records that it has already done	Update many records at once with `MongoTemplate`
Money is stored as a floating-point number, or times are stored in local time	Amounts are a penny wrong. Times are an hour wrong after the clocks change	Use `Money` and `Instant` for all new data	A migration converts the stored values. The business agrees the rounding	Correct the values only when you display them

Event and integration mistakes

Mistake	How you notice	Do now	Fix properly	Never
A change did not send an event	Another team never heard about the change. The audit log shows the change, but `event_publication` has no event	Add the missing `raise(...)` to the entity method. Add the line to the event mapper	Agree with the receiving teams how to send the events they missed. Usually a one-off job sends a correction event for each affected record	Let other teams read our database to catch up
A published event was changed in a way that breaks receivers	Another team's listener fails after our release	Roll back, or put the field back	Send both `V1` and `V2` until every receiving team has moved to `V2`. Then stop sending `V1`	Change a published event without telling the receiving teams
There are messages in the dead-letter queue, or events are not leaving the outbox	An alarm goes off	Find the cause in the logs, using the message's trace ID. Fix it	Move the messages from the dead-letter queue back to the main queue ("redrive"). The inbox skips any that were already processed	Delete dead-letter messages before you understand them. Each one is a change that we have not applied
Our data and another microservice's data do not match	Two microservices show different answers, for example about a payment	Use both audit logs to find which side is right	Add a scheduled job that compares the two and **reports** differences. Make corrections through entity methods, so that they are checked and audited	Add a job that silently overwrites one side with the other side's data
A listener handles repeated or late messages wrongly	The same payment is recorded twice. Or an older payment replaces a newer one	Make sure the listener extends `InboxMessageHandler`	Add a check in the entity method that ignores anything older than what it already has	Assume that each message arrives exactly once and in order
Another microservice's data classes or terms appear in our domain code	Their class names or values (such as `"ELIGIBLE"`) appear outside `client/`	Nothing urgent	Translate them into our own types in the client adapter. Remove their types from everywhere else	Add our own fields to their classes
A call to another microservice has no time limit or circuit breaker	When the other microservice is slow, our requests pile up and our microservice slows down too	Wrap the call in `ResilientCalls`	Add client tests for timeouts and `503`	Retry with no limit. Or retry `4xx` errors
Two microservices always have to change and be released together	Most features need changes in both	Tell the design group	The design group redraws the boundary and records it in an ADR. The feature folder and its data move to the other microservice	Let the two microservices share a database
Code reads another microservice's database	There is a connection string or collection name that belongs to another team	Remove it before release	Use their API or their events	Allow a "temporary" read

Security mistakes

Mistake	How you notice	Do now	Fix properly	Never
An endpoint has no permission check	The architecture test fails. Or a test finds an endpoint that any signed-in user can call	If it is live, follow the security incident process. Add the annotation and release	Add a test that expects `403`. Check the controller's other endpoints	Add the check in the application service instead of the controller
Personal data is in logs, error messages or events	A name, date of birth, National Insurance number or address appears where it is not needed	Remove it and release. If it reached production logs, report it through the data protection process	Use value objects that hide personal data in `toString()`, as `ClaimantDetails` does. Send IDs, not personal details, in events	Decide on your own that it does not matter

9. Never do these

- Write an entity's fields from outside the entity.
- Turn off the version check, or work around it.

- Edit a migration that has run in any environment.
- Call another microservice inside a database transaction.
- Retry a call that changes data in another system.
- Read or write another microservice's database.
- Send to SNS without the outbox.
- Remove or rename a field in a published event version. Publish a new version instead.
- Delete dead-letter messages without investigating them.
- Repair data by writing directly to the database, except in a reviewed migration.
- Release an endpoint without a permission annotation.
- Log personal data.
- Turn off an architecture test.
- Add any of these without design group approval: CQRS (separate models for reading and writing), event sourcing (storing a record as its list of events, not its current state), generic repositories (one repository class shared by all aggregates), or a domain library shared between microservices.

10. Who to ask

You need	Ask	When
To know where some code belongs	A senior engineer, or the weekly design session	On the same day
A new aggregate, a change to what an aggregate contains, a transaction that saves more than one aggregate, or a new link between microservices	The design group, through the design note or an ADR	Before you write code
A change to a published event	The teams that receive it	Before release
Help with a mistake from section 8	The design group	On the same day
A security or personal data problem	The incident process	Immediately

Still to be filled in: design group members, the team channel, the design session time, the platform team contact, and the incident process link.

11. Glossary

Term	Meaning
Aggregate	A group of objects that are always loaded and saved together, as one document
Aggregate root	The top object of an aggregate. All changes go through it (`Claim`)
Anaemic model	A design where entities only hold data, and all logic is in services. Do not use it for Rich features
Application service	The class that runs each use case, step by step (`ClaimService`)
Bounded context	A business area with its own words and its own model. Usually it is one microservice
Business action	One change that the user asks for with one button, such as "close claim". It is one application service method
Circuit breaker	It stops calls to a failing microservice for a short time, after repeated failures
Client adapter	The only class that talks to another microservice or system. It translates their data into our types (`EligibilityClient`)
Dead-letter queue	Where a message goes after it fails 5 times. Someone must investigate it
Domain event	A record that something happened. It is used only inside our microservice (`ClaimClosed`)
Domain service	A rare class for a business calculation that belongs to no single entity (`EntitlementCalculator`)
Entity	A business object with its own ID (`Claim`, `Note`)
Eventual consistency	Other microservices learn about a change a little later, through events
Inbox	The platform's list of processed event IDs. It is used to skip duplicates
Integration event	A versioned event that other microservices receive (`ClaimClosedV1`)
Invariant	A business rule that must be true at all times
Optimistic locking	The version check. A save is refused with `409` if the document changed after it was loaded

Term	Meaning
Outbox	The `event_publication` collection. Events are stored here with the change, before they are sent
Ubiquitous language	The business's own words. Use them in the same way in conversation, design notes, code and tests
Value object	A value with no ID that cannot change. Its contents define it (`Money`)

12. Your first week

1. Read sections 1 to 3. Then explain two things to a colleague: the difference between an entity and a value object, and the difference between an application service and a domain service.
2. Read `Claim`, `Money`, `ClaimTest` and `EntitlementCalculator` in the reference code. On a practice branch, add a rule to `Claim`. Write its test first.
3. Read `ClaimService`, `ClaimController` and `EligibilityClient`, and their tests.
4. Read sections 6 and 7. Run the microservice on your computer. Create a claim and close it. Then look at the `claims`, `notes` and `event_publication` collections.
5. Build the "reopen a claim" example from section 5.5 on a practice branch. Ask a senior engineer to review it against sections 3 and 6.